Reasoning Language Models

Introduction

Contents

1. **What is reasoning?**
   a. Reasoning and Decomposition
2. **Prompting Methods, Inference-Time reasoning**
   a. Chain of Thought prompting
   b. Train of Thoughts
   c. Analogical Prompting
3. **Training for reasoning**
   a. RLHF Recap
   b. Rewards and Bootstrapping, STaR, ORM, PRM
   c. Reinforcement Learning with Verifiable Rewards
   d. DeepSeek-R1?
4. **Test-time compute?**
5. **Conclusion and critical questions**

What is reasoning?

Reasoning and Decomposition

Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems. [1]

Problem 1:
Question: Two trains running in opposite directions cross a man standing on the platform in 27 seconds and 17 seconds respectively and they cross each other in 23 seconds. The ratio of their speeds is:
Options: A) 3/7 B) 3/2 C) 3/88 D) 3/8 E) 2/2

Correct Option: B

Reasoning and Decomposition

Problem 1:
Question: Two trains running in opposite directions cross a man standing on the platform in 27 seconds and 17 seconds respectively and they cross each other in 23 seconds. The ratio of their speeds is:
Options: A) 3/7 B) 3/2 C) 3/88 D) 3/8 E) 2/2
Rationale: Let the speeds of the two trains be x m/sec and y m/sec respectively. Then, length of the first train = 27x meters, and length of the second train = 17 y meters. $(27x + 17y) / (x + y) = 23 \rightarrow 27x + 17y = 23x + 23y \rightarrow 4x = 6y \rightarrow x/y = 3/2$.
Correct Option: B

Figure 1: Example of the reasoning problems in *Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems.* [1]. This paper, published at ACL in 2017, is the first one to use natural language to describe intermediate reasoning steps.

Reasoning and Decomposition

Thinking, Fast and Slow. [2]

1. **System 1 reasoning**: fast, automatic and unconscious; e.g. recognizing faces, taking quick decisions
2. **System 2 reasoning**: slow, deliberate, analytical; e.g. solving maths problems

System 1 is prone to **cognitive biases**, System 2 is **rational**. How do machines pass **from System 1 to System 2?**

Reasoning and Decomposition

Prompting strategies: inference time reasoning

Chain of Thought Prompting

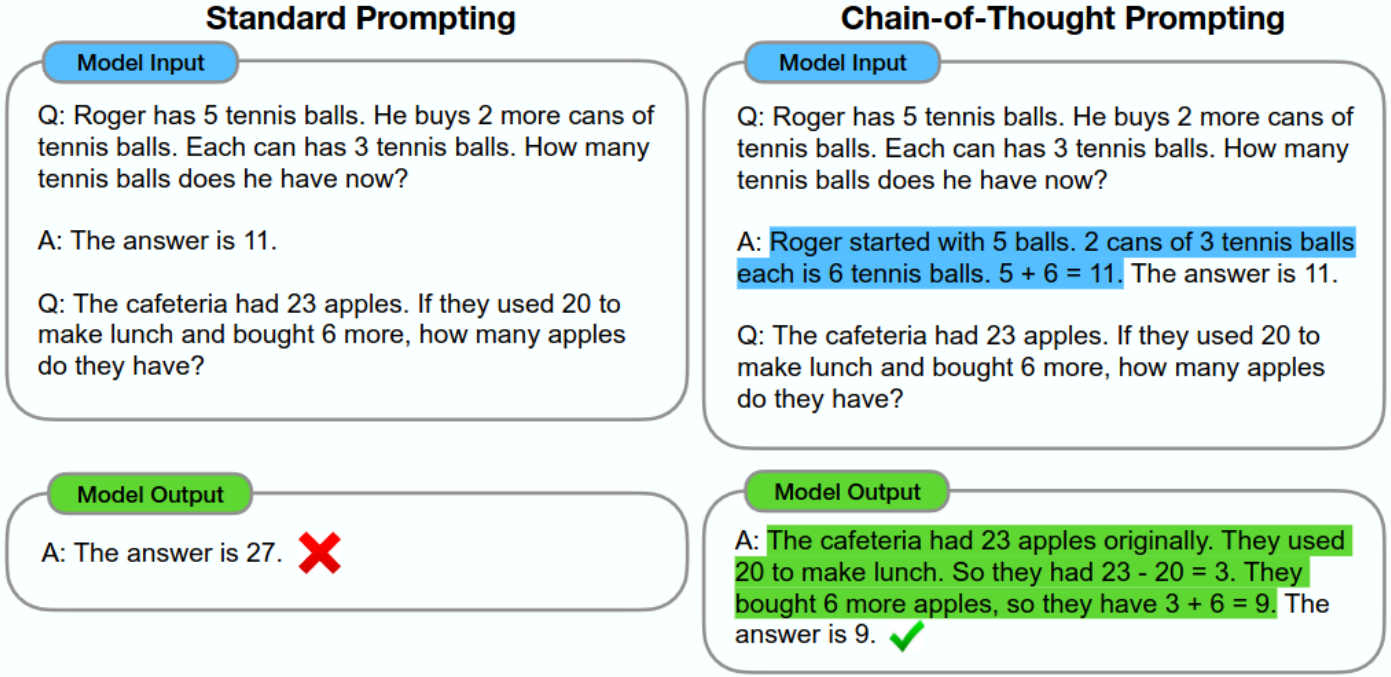
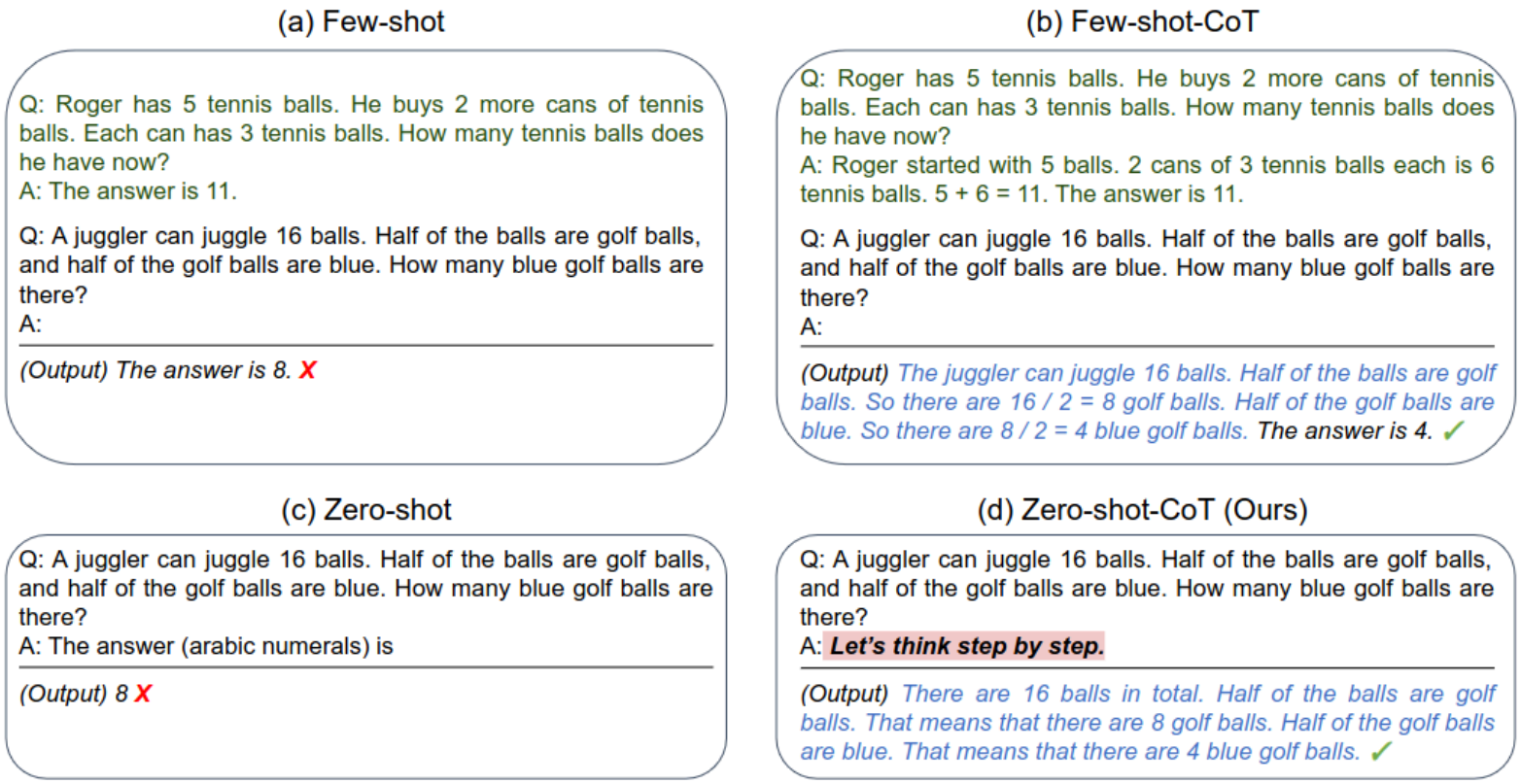


Figure 2 : The 2021 NeurIPS paper by Wei et al. introduced *Chain of Thought (CoT) prompting*, which enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

Chain of Thought prompting



Example inputs and outputs of GPT-3. [3]

Chain of Thought prompting [4]

CoT introduces intermediate reasoning steps $z_1, \ldots, z_n$ between input x and output y:

$$x \to z_1 \to \cdots \to z_n \to y$$

Each thought is generated sequentially:

$$z_i \sim p_\theta^{CoT}(z_i \mid x, z_{1:i-1})$$

The final answer is generated conditioned on the full chain:

$$y \sim p_\theta^{CoT}(y \mid x, z_{1:n})$$

Chain of Thought prompting [4]

Summary: reasoning steps and the answer are sampled as a single continuous sequence:

$$[z_1, \ldots, z_n, y] \sim p_\theta^{CoT}(z_1, \ldots, z_n, y \mid x)$$

The **decomposition** of thoughts (phrase, sentence, paragraph, ...) is left ambiguous.

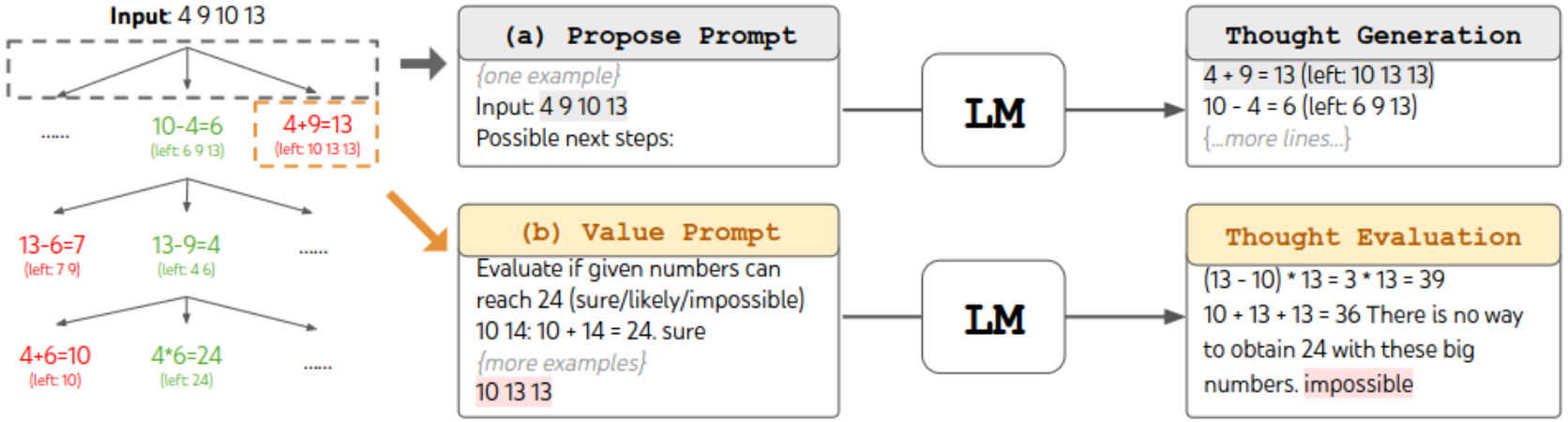
Tree of Thoughts

Deliberate Problem Solving with Large Language Models [4]

1. How to decompose the intermediate process into thought steps
2. How to generate potential thoughts from each state
3. How to heuristically evaluate states
4. What search algorithm to use

Tree of Thoughts

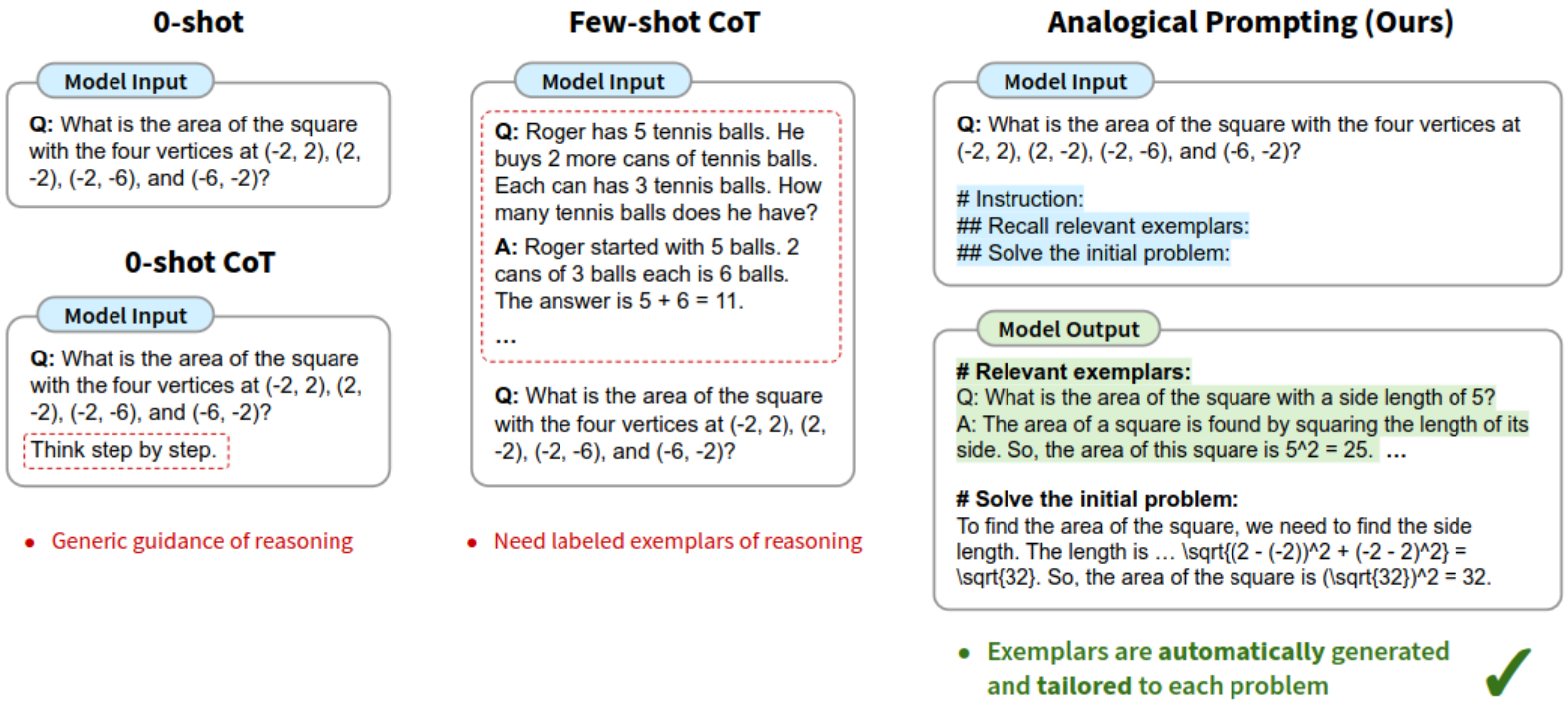
Deliberate Problem Solving with Large Language Models [4]



Example of ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation.

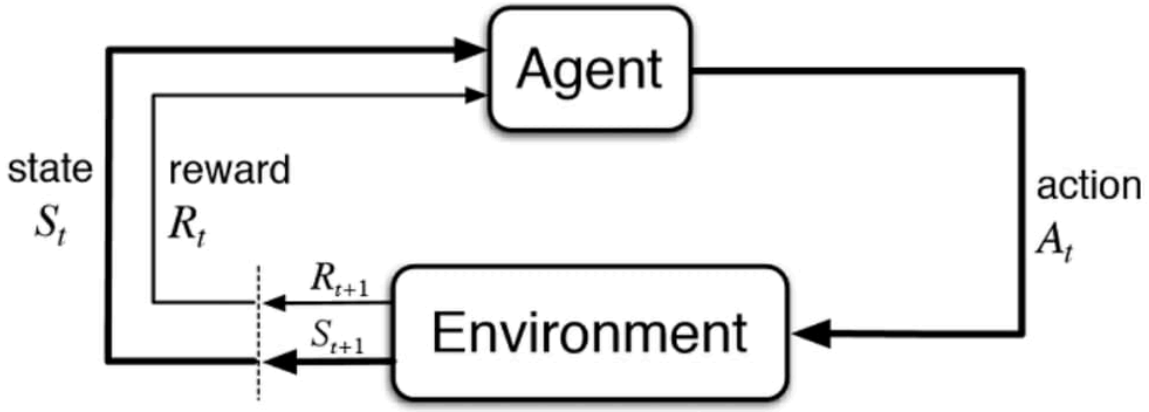
Analogical Prompting

Large Language Models as Analogical Reasoners [5]



Reinforcement Learning Recap

Reinforcement Learning Recap



In reinforcement learning, an agent learns from the environment by interacting with it through trial and error and receiving rewards as feedback for performing actions.

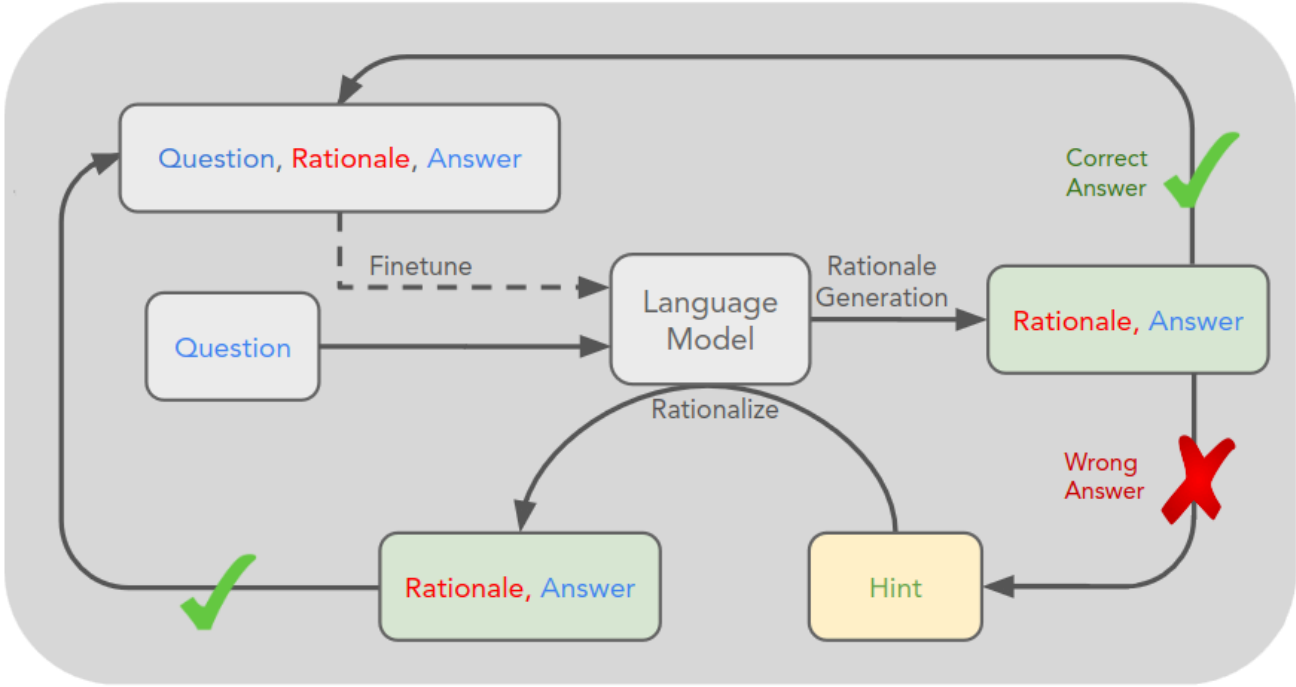
The agent does not know the environment beforehand (black box): this is why it is a **learning** process.

Reinforcement Learning

- **Environment**, with which the agent interacts, outputs **observations**
- Agent makes **decisions**
- Agent gets new **observations**
- Agent gets a **rewards** based on how good (or bad) the actions were.

Training for Reasoning

STaR: Bootstrapping Reasoning with Reasoning [6]




```
Q: What can be used
to carry a small dog?
Answer Choices:
(a) swimming pool
(b) basket
(c) dog show
(d) backyard
(e) own home
A: The answer must be
something that can be
used to carry a small
dog. Baskets are
designed to hold things.
Therefore, the answer
is basket (b).
```

An overview of STaR and a STaR-generated rationale on CommonsenseQA.Fine-tuning outer loop is the dashed line. Questions and ground truth answers in the dataset, rationales generated using STaR. [4]

STaR: Bootstrapping Reasoning With Reasoning [6]

Input

- Pretrained LLM **M**
- Dataset:

$$D = \{(x_i, y_i)\}$$

- Few-shot rationale examples:

$$P = \{(x_i^p, r_i^p, y_i^p)\}$$

STaR: Bootstrapping Reasoning With Reasoning [6]

Iterative loop

1. Generate rationale:

$$M(x_i, P) \rightarrow (\hat{r}_i, \hat{y}_i)$$

2. Filter:

$$\hat{y}_i = y_i$$

3. Fine-tune on:

$$(x_i, \hat{r}_i, y_i)$$

4. Repeat

RL in STaR

The LLM first generates a rationale r and then predicts an answer y.

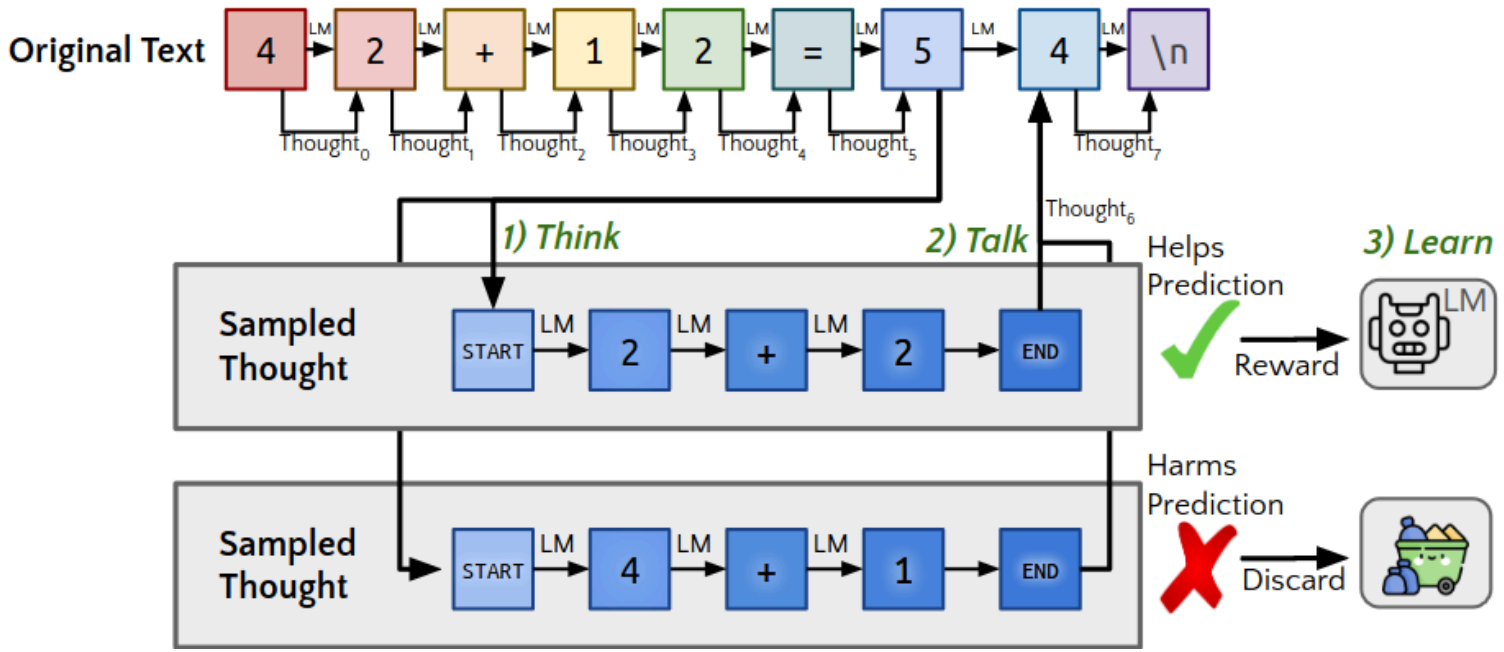
$$p_M(y|x) = \sum_r p(r|x)p(y|x,r)$$

STaR approximates maximizing:

$$J(M) = \sum_i \mathbb{E}_{\hat{r}_i,\hat{y}_i}[\mathbb{1}(\hat{y}_i = y_i)]$$

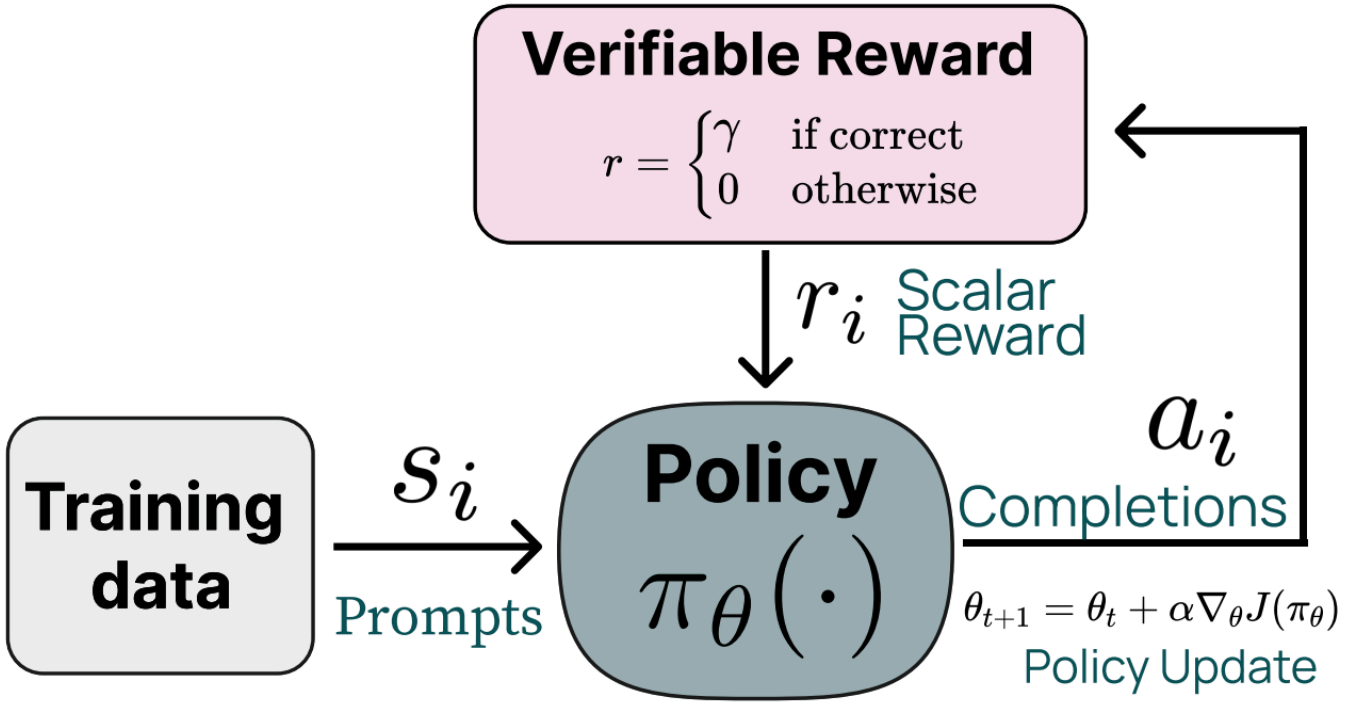
where the reward is **1 only for correct answers**.

Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking [5]



Reinforcement Learning with Verifiable Rewards (RLVR)

Tülu 3: Pushing Frontiers in Open Language Model Post-Training. [8]



RLVR

The model is trained on tasks for which the correctness can be (easily) verified, either in an exact, mathematical way (formal proofs) or through executables (e.g. unit tests).

There is no uniform way of defining a task as verifiable; for example, mathematical proofs are (intuitively) among the most robustly verifiable tasks.

RLVR: Verifying Function

Given a question x, a generated solution trajectory is:

$$\tau = (r_1, \ldots, r_T, y)$$

where $r_{1:T}$ denotes the reasoning trace and y the final answer.

The verifier V_ϕ assigns a score to a candidate solution:

$$V_\phi(x, \tau) \rightarrow s \in [0, 1]$$

RLVR: Verifying Function

Training data consists of solution trajectories with correctness labels:

$$\mathcal{D}_V = \{(x_i, \tau_i, c_i)\}_{i=1}^{N}, \quad c_i \in \{0, 1\}$$

The verifier is optimized

GRPO: critic-free policy optimization

GRPO (Shao et al., 2024) removes the value network of PPO and replaces
$\hat{A}_t$ with a **group-relative** z-score of the terminal reward:

$$\hat{A}_{i,t} = \frac{r_i - \mathrm{mean}(\{r_1, \ldots, r_G\})}{\mathrm{std}(\{r_1, \ldots, r_G\})}$$

GRPO: critic-free policy optimization (2)

Objective (token-level ratio, sequence-level normalization):

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q,\{o_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t},1-\varepsilon,1+\varepsilon)\hat{A}_{i,t}\right)\right] - \beta\, D_{\text{KL}}(\pi_\theta\|\pi_{\text{ref}})$$

with importance ratio $\rho_{i,t} = \dfrac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}$.

No critic → half the memory/compute of PPO; the "baseline" is the group mean itself.

Dr.GRPO: removing the normalization bias

Dr.GRPO (Liu et al., 2025) shows both normalizers are **not** part of the true policy-gradient estimator — they're optimization artifacts — and simply drops them:

$$\hat{A}_i = r_i - \mathrm{mean}(\{r_1, \ldots, r_G\}) \qquad (\text{no } / \text{ std})$$

$$\mathcal{J}_{\text{Dr.GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{\sum_i |o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\left(\rho_{i,t}\hat{A}_i,\ \mathrm{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\right)\right]$$

Loss is aggregated with a **global** token count $\sum_i |o_i|$ instead of per-sample averaging — an unbiased Monte-Carlo estimate of the true

DAPO: four fixes for long-CoT stability

DAPO (Yu et al., 2025) targets the same failure modes with a different recipe, used to train an open R1-scale reasoner:

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a),\{o_i\}\sim\pi_{\theta_{\text{old}}}} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}})\hat{A}_{i,t}\right) \right]$$

$$\text{s.t.} \quad 0 < \left| \{\, o_i : \text{is_equivalent}(a, o_i) \,\} \right| < G$$

- **Clip-Higher**: $\varepsilon_{\text{low}}=0.20,\ \varepsilon_{\text{high}}=0.28$ — asymmetric bound lets low-probability "surprising" correct tokens get reinforced more, mitigating entropy collapse.
- **Dynamic Sampling**: resample any group with all-correct or all-

DeepSeek-R1

Questions?

References -- TODO: Correct Bibliography

[1] Ling, Wang, Yogatama, Dani, Dyer, Chris, and Blunsom, Phil. "Program Induction by Rationale Generation : Learning to Solve and Explain Algebraic Word Problems.", 2017.

[2] Kahneman, Daniel. Thinking, Fast and Slow. Farrar, Straus and Giroux, 2011.

[3] Kojima et al., Large Language Models are Zero-Shot Reasoners, NeurIPS 2022.

[4] Yao et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.

[5] Yasunaga et al. Large Language Models as Analogical Reasoners. ICLR 2024.

[6] Zelikman et al. STaR: Bootstrapping Reasoning With Reasoning. NeurIPS 2022.

[7] Wen et al. Reinforcement Learning with Verifiable Rewards Implicitly Incentivizes Correct Reasoning in Base LLMs.

[8] Lambert et al. Tülu 3: Pushing Frontiers in Open Language Model Post-Training. 2025